Video RAG Retrieval Project

https://github.com/aiyshwary/Video-RAG-Retrieval

Python FAISS OpenCLIP ViT-GPT2 Sentence Transformers RAG

OVERVIEW

A semantic video search system that extracts frames, generates scene captions using ViT-GPT2, and embeds them with SentenceTransformers and OpenCLIP into a dual FAISS index, enabling natural language queries to retrieve the most relevant video scenes with timestamps.

IMPACT

Built a Visual RAG system that makes unstructured video content semantically searchable: a text query retrieves the most relevant scenes with timestamps by matching against dual text and image embedding indexes via cosine similarity.

TECHNICAL WALKTHROUGH

One-line summary
“I built a Visual Retrieval-Augmented Generation (RAG) system that retrieves relevant video scenes based on a user’s text prompt by combining vision models and vector similarity search.”

Problem statement
“Searching videos using text is hard because video content is unstructured.
Traditional keyword search doesn’t understand what’s actually happening inside a video. My project solves this by converting both video scenes and user queries into embeddings and retrieving the most relevant video segments.”

High-level architecture

“The system has three main components

i) Scene understanding using vision models

ii) Vector storage for text and image embeddings

iii) Similarity-based retrieval using VDMS”

iv) Video → Scene Description + Frames → Embeddings → Vector DB

v) → Retrieval → Video Clips

Input & preprocessing
“The input to the system is one or more videos. Each video is processed to

extract

i) Scene-level textual descriptions using open-source vision models like

ii) Video-LLaMA

iii) Key video frames along with metadata like frame number, FPS, and

iv) timestamp”*

Vision understanding
“I use vision-language models to understand the video content. These models analyze video scenes and generate natural language descriptions that summarize what’s happening in each scene.”
“For example, a scene might be described as ‘a person walking into a store and picking up an item’.”

Embedding strategy (very important for
interviewers)
“To support efficient retrieval, I store two types of embeddings in separate vector stores:”

Text embeddings
Model: SentenceTransformer – all-MiniLM-L6-V2 / L12-V2

Used for

i) Scene descriptions

ii) User queries

iii) “This converts textual descriptions into dense vectors that capture semantic

iv) meaning.”

v) Image embeddings

vi) Model: OpenCLIP – ViT-g-14

vii) Video frames

viii) “This helps capture visual semantics like objects, actions, and context directly

ix) from frames.”

Vector storage & retrieval
“Both text and image embeddings are stored in VDMS, which allows fast vector similarity search. I maintain separate vector collections for text and image embeddings but link them using video metadata.”

Stored metadata includes

i) Video name

ii) Scene ID

iii) Frame number

iv) Timestamp

Query flow
“When a user enters a text prompt:”

Prompt is converted into a text embedding

Vector similarity search is performed against

i) Scene text embeddings

ii) (Optionally) image embeddings

The system retrieves

i) Most relevant scenes

ii) Corresponding frames and timestamps

Final output

i) Scene description

ii) Exact time range to jump to in the video

iii) question)

iv) “Text embeddings capture semantic meaning, while image embeddings capture

v) visual features. Using both improves retrieval accuracy because some queries are better answered by visual similarity and others by textual context.”

Example

i) “‘a red car’ → image embeddings

ii) ‘a tense conversation’ → text embeddings”

iii) Technical depth

iv) Efficient vector search using cosine similarity

v) Decoupling embeddings for scalability

vi) Model choice based on speed vs accuracy

vii) Frame sampling using FPS to avoid redundancy

viii) Closing line (impact)

ix) “This approach makes video content searchable, explainable, and scalable, and

x) it can be extended to surveillance, media indexing, or enterprise video search.”

xi) Short version

xii) “I built a Visual RAG system that enables semantic search over videos. It uses

xiii) vision models to generate scene descriptions and extract frames, converts them into text and image embeddings using SentenceTransformers and OpenCLIP, stores them in VDMS, and retrieves relevant video segments based

xiv) on user queries.”

KEY DETAILS

1. Extracts frames at a configurable FPS and generates natural-language scene captions using ViT-GPT2 image captioning.
2. Computes text embeddings with SentenceTransformers (MiniLM) and image embeddings with OpenCLIP for dual-modality indexing.
3. Stores embeddings and metadata in two parallel FAISS indexes supporting independent text or image-based retrieval.
4. Query interface accepts a natural language string and returns top-k matching scenes with frame references and timestamps.
5. Modular architecture (preprocessing, embeddings, storage, retrieval) designed for extension to production vector databases.
6. FAISS indexes use IndexFlatIP with L2-normalized vectors for cosine-similarity search; OpenCLIP defaults to ViT-g-14 pretrained on LAION-2B with automatic GPU/CPU selection.
7. Includes a pytest suite and a self-contained demo that synthesizes a 30-frame test video with OpenCV to run the full index-and-query pipeline without needing a real video file.